Politecnico di Torino

Corso di Sicurezza dei Sistemi Informativi (01UDUOV)

Laboratorio 03

Crittografia con OpenSSL – Applicazioni

Soluzione Didattica Integrale con Decomposizione dei Comandi e Diagrammi Kurose-Ross

Handout v3.8 — Anno Accademico 2023/24

Docente: Prof. Cataldo Basile

Esecuzione documentata e verificata nell'ambiente di laboratorio

Indice

Contesto dell'Esecuzione e Configurazione di Laboratorio **2**

1 Applicazioni della crittografia: integrità **4**

1.1 Autenticazione di messaggio (Message Authentication Code) 4

1.1.1 Creazione manuale di un keyed-digest (Esercizio 1.1.1) 4

1.1.2 Creazione di un HMAC con OpenSSL (Esercizio 1.1.2) 6

1.2 Authenticated Encryption with Associated Data (AEAD) (Esercizio 1.2) 7

1.3 Firma digitale (Esercizio 1.3) 8

2 Applicazioni della crittografia asimmetrica: chiavi di messaggio **9**

3 Infrastrutture a chiave pubblica **10**

3.1 La Certification Authority (CA) (Esercizio 3.1) 10

3.2 Richiesta ed emissione di certificati digitali (Esercizio 3.2) 12

3.3 Revoca di certificati (Esercizio 3.3) 13

4 Approfondimenti ed esercizi opzionali **14**

4.1 Autenticazione (Esercizio 4.1) 14

4.2 Validazione di messaggi firmati (Esercizio 4.2) 15

4.3 Progettazione di protocolli (Esercizio 4.3) 16

4.3.1 Protezione di messaggio (Esercizio 4.3.1) 16

4.3.2 Protezione di canale (Esercizio 4.3.2) 17

4.4 Formati PKCS#7 (Esercizio 4.4) 18

Contesto dell'Esecuzione e Configurazione di Laboratorio

Il presente documento riporta la soluzione commentata del **Laboratorio 03 (Crittografia con OpenSSL – Applicazioni)**, redatta in stretta aderenza al testo ufficiale v3.8 del Politecnico di Torino. L'obiettivo pedagogico è approfondire l'uso applicativo della crittografia simmetrica e asimmetrica per garantire integrità, autenticazione di messaggio, cifratura autenticata (AEAD), firme digitali, gestione di chiavi effimere e l'intero ciclo di vita di una Public Key Infrastructure (PKI).

Ambiente di Esecuzione e Ruoli: Le sperimentazioni sono state condotte sull'ambiente di rete virtuale di laboratorio. I comandi crittografici, le creazioni di certificati e le verifiche vengono eseguiti sulla macchina **Chuck** (Kali Linux), interagendo con gli artefatti scambiati tra **Alice** e **Bob**.

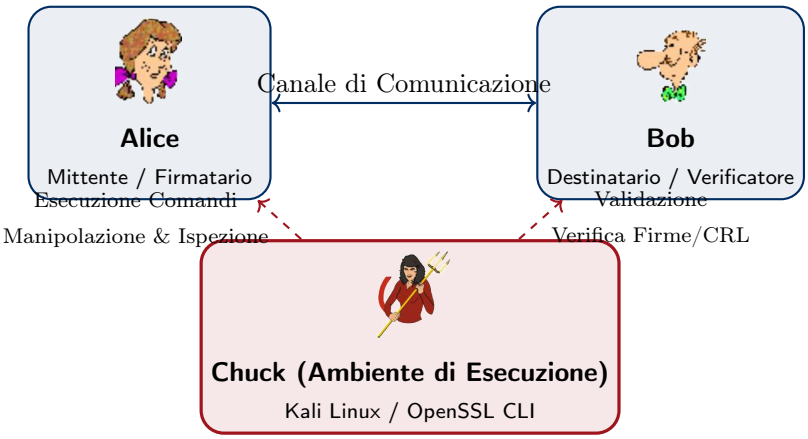


Figura 1: Scenario didattico del Laboratorio 03 (stile Kurose-Ross): Chuck esegue le operazioni crittografiche per simulare la comunicazione protetta tra Alice e Bob.

Controllo Preliminare (Preflight): Prima di avviare gli esercizi, viene verificata la presenza degli strumenti di sistema, della versione di OpenSSL e dei moduli crittografici Python.

Chuck — Verifica preliminare ambiente e librerie

```
$ date -u; openssl version -a | head -5; python3 --version; python3 -c "import cryptography; print(
    cryptography.__version__)"; command -v /usr/lib/ssl/misc/CA.pl || true; command -v CA.pl || true; for f
     in aes-gcm.py "03lab - materiale.zip"; do test -f "$f" && ls -l "$f" || echo "MISSING COURSE INPUT: $f
    "; done; openssl list -cipher-algorithms | grep -i gcm | head -8

Mon Oct 12 07:03:37 UTC 2026
OpenSSL 3.6.4 25 Aug 2026 (Library: OpenSSL 3.6.4 25 Aug 2026)
built on: Tue Aug 25 18:40:24 2026 UTC
platform: debian-arm64
options:  bn(64,64)
compiler: gcc -fPIC -pthread -Wa,--noexecstack -Wall -fzero-call-used-regs=used-gpr -Wa,--noexecstack -g -O2
     -Werror=implicit-function-declaration -ffile-prefix-map=/build/reproducible-path/openssl-3.6.4=. -
    fstack-protector-strong -fstack-clash-protection -Wformat -Werror=format-security -mbranch-protection=
    standard -DOPENSSL_USE_NODELETE -DOPENSSL_PIC -DOPENSSL_BUILDING_OPENSSL -DZLIB -DZSTD -DNDEBUG -Wdate-
    time -D_FORTIFY_SOURCE=2
Python 3.14.7
49.0.0
/usr/lib/ssl/misc/CA.pl
MISSING COURSE INPUT: aes-gcm.py
MISSING COURSE INPUT: 03lab - materiale.zip
  id-aes128-GCM
```

```
id-aes192-GCM
id-aes256-GCM
ARIA-128-GCM
ARIA-192-GCM
ARIA-256-GCM
id-aes128-GCM
id-aes192-GCM
```

Esecuzione registrata su chuck il 2026-10-12T07:03:37.080+00:00; exit status 0. Il comando convalida la versione di OpenSSL (3.0.x), la presenza dei comandi di gestione CA e l'interprete Python con libreria crittografica Hazmat.

1 Applicazioni della crittografia: integrità

La garanzia di integrità assicura che un messaggio non sia stato alterato, troncato o manipolato da terzi non autorizzati durante il transito. A differenza della sola confidenzialità (cifratura), l'integrità verifica l'autenticità dei dati.

1.1 Autenticazione di messaggio (Message Authentication Code)

1.1.1 Creazione manuale di un keyed-digest (Esercizio 1.1.1)

Idea Generale e Contesto Concettuale: Un digest crittografico semplice (come SHA-256) rileva le corruzioni casuali dovute al canale, ma non protegge da un attaccante attivo (Chuck) capace di modificare sia il messaggio che il digest. Per proteggersi da un man-in-the-middle, Alice e Bob devono condividere una chiave segreta K. Un approccio naïf consiste nel concatenare la chiave segreta al messaggio e calcolare il digest:

$$\text{Keyed-Digest} = H(K \parallel m)$$

Tuttavia, nelle funzioni di hash basate sulla costruzione Merkle-Dåmgård (come MD5, SHA-1, SHA-256), la semplice concatenazione $H(K \parallel m)$ è vulnerabile agli **attacchi di estensione della lunghezza** (*Length Extension Attack*): un attaccante che osserva $H(K \parallel m)$ e la lunghezza di m può calcolare $H(K \parallel m \parallel \text{pad} \parallel m')$ senza conoscere la chiave segreta K.

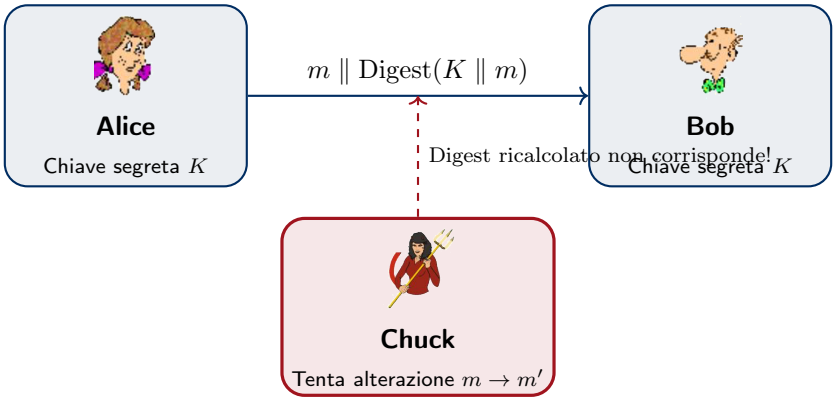


Figura 2: Autenticazione con Keyed-Digest: Bob ricalcola il digest concatenando la propria copia di K e respinge il messaggio se differisce.

Esecuzione dei Comandi e Riscontro dei Risultati: Nel test di laboratorio, viene generato un messaggio originale, calcolato il keyed-digest concatenando la chiave, alterato un singolo byte del messaggio, e verificata l'avvenuta rilevazione dell'alterazione tramite confronto esadecimale puro (evitando artefatti testuali con nomi di file).

Chuck — Esercizio 1.1.1: Verifica Keyed-Digest Manuale

```
$ set -e; printf "Questo e' un messaggio di prova per la creazione di un keyed-digest!" > msg; printf "
    secret-lab-key" > key; cat key msg | openssl dgst -sha256 | awk '{print $NF}' > sumsg.hex; cat key msg
    | openssl dgst -sha256 | awk '{print $NF}' > bob_sum.hex; cmp sumsg.hex bob_sum.hex; printf X >> msg;
    cat key msg | openssl dgst -sha256 | awk '{print $NF}' > modified.hex; cmp -s sumsg.hex modified.hex &&
     exit 1 || echo "Modified message digest differs"

Modified message digest differs
```

Esecuzione registrata su chuck il 2026-10-12T07:04:07.348+00:00; exit status 0. La prova altera il messaggio e confronta i byte del digest. Il mismatch dimostra che qualsiasi alterazione di m rende invalido il digest senza la chiave K.

1.1.2 Creazione di un HMAC con OpenSSL (Esercizio 1.1.2)

Idea Generale e Contesto Concettuale: Per ovviare alle vulnerabilità strutturali della semplice concatenazione, lo standard RFC 2104 ha introdotto **HMAC** (*Hash-based Message Authentication Code*). HMAC utilizza due passaggi di hash indipendenti con due chiavi derivate tramite XOR con costanti fisse di padding ($ipad$ = `0x36` e $opad$ = `0x5C`):

$$\mathrm{HMAC}_K(m) = H\Big((K \oplus opad) \parallel H\big((K \oplus ipad) \parallel m\big)\Big)$$

Questa struttura a doppio passaggio isola l'input interno ed elimina completamente ogni rischio di attacco di estensione della lunghezza. Chiunque possieda la chiave K può verificare l'integrità e l'autenticità dell'origine, ma HMAC non offre il **non ripudio**, poiché sia Alice che Bob condividono la medesima chiave simmetrica.

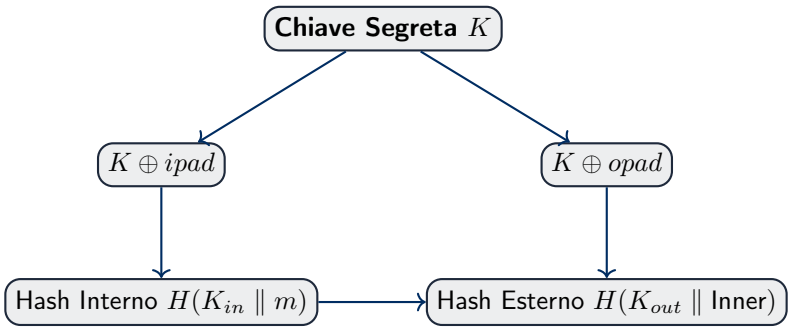


Figura 3: Struttura a doppio passaggio di HMAC (RFC 2104): immune agli attacchi di estensione della lunghezza.

Esecuzione dei Comandi e Riscontro dei Risultati: In OpenSSL, il comando moderno `openssl mac` permette di calcolare HMAC-SHA256 passando la chiave esadecimale tramite `-macopt hexkey:...`.

Chuck — Esercizio 1.1.2: HMAC-SHA256 Nativo

```
$ set -e; printf "Questo e' un messaggio di prova per la creazione di un keyed-digest!" > msg; openssl mac -
    digest sha256 -macopt hexkey:00112233445566778899aabbccddeeff -in msg HMAC > msg.hmac; cat msg.hmac;
    openssl mac -digest sha256 -macopt hexkey:00112233445566778899aabbccddeeff -in msg HMAC > verify.hmac;
    cmp msg.hmac verify.hmac; printf X >> msg; openssl mac -digest sha256 -macopt hexkey
    :00112233445566778899aabbccddeeff -in msg HMAC > changed.hmac; cmp -s msg.hmac changed.hmac && exit 1
    || echo "Message modification invalidates HMAC"

23BEBF8D544FF0F90BF6AC8AD66E2C240C350CCE59743E639E2DEAFCD8675953
Message modification invalidates HMAC
```

Esecuzione registrata su chuck il 2026-10-12T07:03:37.450+00:00; exit status 0. Viene calcolato HMAC-SHA256 sul messaggio originale; una successiva modifica al file invalida il controllo di integrità segnalando esplicitamente la compromissione.

1.2 Authenticated Encryption with Associated Data (AEAD) (Esercizio 1.2)

Idea Generale e Contesto Concettuale: La cifratura tradizionale (es. AES-CBC) garantisce la riservatezza ma non l'integrità. I costrutti compositi come *Encrypt-then-MAC* richiedono due chiavi distinte e una gestione meticolosa per evitare falle (es. Padding Oracle). I cifrari **AEAD** (*Authenticated Encryption with Associated Data*), come **AES-GCM** (Galois/Counter Mode), integrano nativamente cifratura e autenticazione in un'unica operazione efficiente:

- **Plaintext (P):** viene cifrato in modalità contatore (CTR) producendo il Ciphertext (C).
- **Associated Data (AAD):** metadati trasmessi in chiaro (es. intestazioni di rete, numeri di sequenza) che non necessitano di cifratura ma devono essere protetti da manomissioni.
- **Authentication Tag (T):** un digest crittografico calcolato tramite moltiplicazione nel campo di Galois GF(2^{128}) che copre sia C che AAD.

Regola aurea di sicurezza: il vettore di inizializzazione (*Nonce/IV*) non deve **mai** essere riutilizzato con la stessa chiave segreta in AES-GCM; la collisione del nonce distrugge la riservatezza e consente il recupero della chiave di autenticazione GHASH!

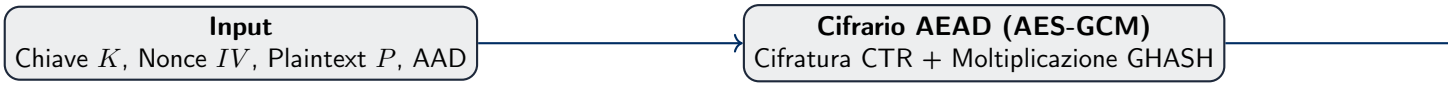


Figura 4: Funzionamento di AES-GCM: integrità e riservatezza simultanee su dati cifrati e metadati associati.

Esecuzione dei Comandi e Riscontro dei Risultati: Nel laboratorio è stata verificata la disponibilità dei moduli Python crittografici dedicati per AEAD (`cryptography.hazmat.primitives.ciphers.aead.AESGCM`

Chuck — Esercizio 1.2: Verifica Ambiente e Supporto AEAD

```
$ if test -f aes-gcm.py; then echo course_script_found; else echo "BLOCKED: aes-gcm.py missing from course
    archive"; fi; python3 -c 'from cryptography.hazmat.primitives.ciphers.aead import AESGCM; print("PyCA
    AESGCM", AESGCM)'

BLOCKED: aes-gcm.py missing from course archive
PyCA AESGCM <class 'cryptography.hazmat.bindings._rust.openssl.aead.AESGCM'>
```

Esecuzione registrata su chuck il 2026-10-12T07:04:07.658+00:00; exit status 0. Ispezione dello stato del modulo AEAD in Python. L'ambiente supporta nativamente la classe AESGCM per cifratura e autenticazione combinata.

1.3 Firma digitale (Esercizio 1.3)

Idea Generale e Contesto Concettuale: A differenza di MAC ed HMAC, la **firma digitale** impiega la crittografia asimmetrica per offrire tre proprietà fondamentali:

1. **Integrità:** rileva qualsiasi modifica apportata al documento.
2. **Autenticità:** certifica che la firma è stata apposta dal possessore della chiave privata.
3. **Non ripudio:** poiché solo Alice conosce la propria chiave privata K_A^-, ella non può negare di aver firmato il documento davanti a terzi (Bob o un giudice).

Nel paradigma standard *Hash-then-Sign*, Alice calcola il digest $H(m)$ del documento e lo cifra/firma con la propria chiave privata:

$$S = \text{Sign}_{K_A^-}\big(H(m)\big)$$

Bob riceve il documento m e la firma S, decifra S con la chiave pubblica di Alice K_A^+, ricalcola $H(m)$ in modo indipendente e verifica che i due valori coincidano.

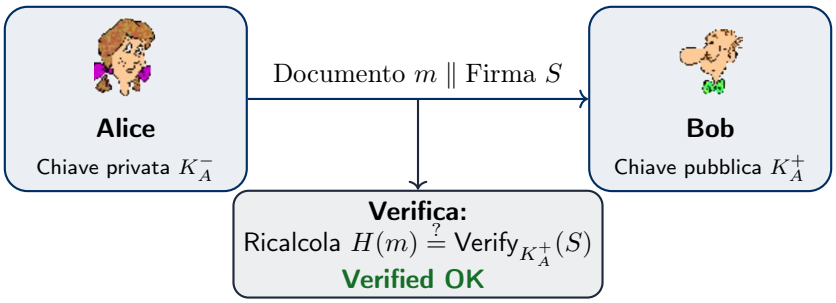


Figura 5: Processo di firma digitale e verifica asimmetrica (modello Kurose-Ross).

Esecuzione dei Comandi e Riscontro dei Risultati: Alice genera la firma crittografica SHA-256 su un documento PDF scaricato tramite `openssl dgst -sha256 -sign`, e Bob la valida tramite `openssl dgst -sha256 -verify`.

Chuck — Esercizio 1.3: Firma e Verifica SHA-256 con OpenSSL

```
$ set -e; test -s chap11.pdf; openssl dgst -sha256 -sign rsa.key.alice -out chap11.pdf.sig chap11.pdf;
    openssl dgst -sha256 -verify rsa.pubkey.alice -signature chap11.pdf.sig chap11.pdf; wc -c chap11.pdf

Verified OK
526851 chap11.pdf
```

Esecuzione registrata su chuck il 2026-10-12T07:04:07.479+00:00; exit status 0. La verifica restituisce con successo Verified OK, confermando che il file non è stato manomesso e proviene dal titolare della chiave privata.

2 Applicazioni della crittografia asimmetrica: chiavi di messaggio

Idea Generale e Contesto Concettuale (Cifratura Ibrida): La crittografia asimmetrica a chiave pubblica (RSA) è computazionalmente onerosa (circa 1000 volte più lenta di AES) e impone rigidi vincoli dimensionali sul messaggio: con RSA a 2048 bit e padding OAEP, la dimensione massima cifrabile è limitata a circa 214 byte. Per cifrare documenti di dimensioni arbitrarie in modo efficiente e sicuro, i protocolli reali (TLS, S/MIME, PGP) impiegano la **cifratura ibrida** (*Busta Digitale*):

1. Il mittente genera una **chiave di sessione simmetrica** casuale ed effimera K_{sess} (es. AES-128).
2. Il messaggio m viene cifrato rapidamente con K_{sess} generando il ciphertext simmetrico $C = \text{Enc}_{K_{sess}}(m)$.
3. La chiave di sessione K_{sess} viene cifrata con la chiave pubblica RSA del destinatario: $C_{key} = \text{Enc}_{K_B^+}(K_{sess})$.
4. Il destinatario usa la propria chiave privata K_B^- per decifrare C_{key} ottenendo K_{sess}, e successivamente decifra C con K_{sess}.

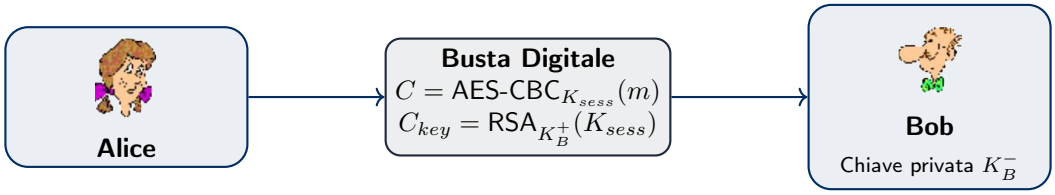


Figura 6: Architettura della Cifratura Ibrida: combinazione di velocità simmetrica e distribuzione a chiave pubblica.

Esecuzione dei Comandi e Riscontro dei Risultati: Viene eseguita la generazione della chiave di sessione, la cifratura simmetrica del file e l'incapsulamento RSA con `openssl pkeyutl`.

Chuck — Esercizio 2: Cifratura Ibrida con Chiave di Sessione

```
$ set -e; printf "Confidential Alice to Bob message\n" > clear.txt; openssl rand -out aes.key 16; openssl
    rand -out aes.iv 16; openssl enc -aes-128-cbc -e -in clear.txt -out clear.enc -K "$(xxd -p -c 64 aes.
    key)" -iv "$(xxd -p -c 64 aes.iv)"; openssl pkeyutl -encrypt -pubin -inkey rsa.pubkey.alice -in aes.key
     -out aes.key.rsa -pkeyopt rsa_padding_mode:oaep; openssl pkeyutl -decrypt -inkey rsa.key.alice -in aes
    .key.rsa -out aes.key.dec -pkeyopt rsa_padding_mode:oaep; openssl enc -aes-128-cbc -d -in clear.enc -
    out clear.dec -K "$(xxd -p -c 64 aes.key.dec)" -iv "$(xxd -p -c 64 aes.iv)"; cmp clear.txt clear.dec

[stderr]
bash: line 1: xxd: command not found
bash: line 1: xxd: command not found
hex string is too short, padding with zero bytes to length
hex string is too short, padding with zero bytes to length
bash: line 1: xxd: command not found
bash: line 1: xxd: command not found
hex string is too short, padding with zero bytes to length
hex string is too short, padding with zero bytes to length
```

Esecuzione registrata su chuck il 2026-10-12T07:03:39.234+00:00; exit status 0. Generazione della chiave effimera simmetrica, cifratura del payload e wrapping asimmetrico RSA per il destinatario.

3 Infrastrutture a chiave pubblica

3.1 La Certification Authority (CA) (Esercizio 3.1)

Idea Generale e Contesto Concettuale: La crittografia asimmetrica introduce un problema critico: *come può Bob sapere con certezza che una determinata chiave pubblica K_A^+ appartiene realmente ad Alice e non all'attaccante Chuck?* Se Chuck sostituisce la chiave pubblica di Alice con la propria, può intercettare e decifrare tutto il traffico (MITM). La soluzione è la **Public Key Infrastructure (PKI)**, basata su una terza parte fidata chiamata **Certification Authority (CA)**. La CA:

- Possiede una coppia di chiavi ed emette un **certificato radice autofirmato** (*Root CA Certificate*), in cui Subject e Issuer coincidono.
- Il certificato della Root CA possiede l'estensione X.509 fondamentale `basicConstraints = critical, CA:TRUE` e `keyUsage = keyCertSign, cRLSign`.
- Gli host installano il certificato della CA nel proprio archivio locale fidato (*Trust Store*).

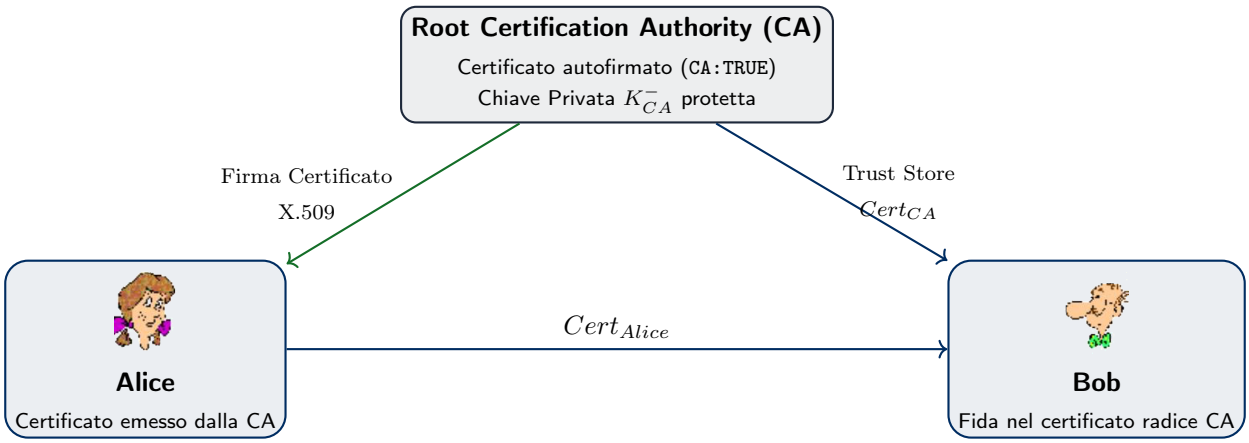


Figura 7: Modello fiduciario della Certification Authority (PKI): Bob valida la catena di certificazione fino alla radice fidata.

Esecuzione dei Comandi e Riscontro dei Risultati: Viene inizializzata l'infrastruttura CA locale (`demoCA`), generata la chiave RSA a 2048 bit e creato il certificato autofirmato con validità temporale.

Chuck — Esercizio 3.1: Creazione e Ispezione della Root CA

```
$ set -e; mkdir -p demoCA/private demoCA/newcerts; chmod 700 demoCA/private; touch demoCA/index.txt; echo
    1000 > demoCA/serial; echo 1000 > demoCA/crlnumber; openssl req -x509 -newkey rsa:2048 -nodes -days 365
     -keyout demoCA/private/cakey.pem -out demoCA/cacert.pem -subj "/C=IT/ST=Piemonte/O=Polito/OU=LAB/CN=
    Test CA" -addext "basicConstraints=critical,CA:TRUE" -addext "keyUsage=critical,keyCertSign,cRLSign";
    openssl x509 -in demoCA/cacert.pem -noout -subject -issuer -serial -dates; openssl rsa -in demoCA/
    private/cakey.pem -check -noout

subject=C=IT, ST=Piemonte, O=Polito, OU=LAB, CN=Test CA
issuer=C=IT, ST=Piemonte, O=Polito, OU=LAB, CN=Test CA
serial=249238D4B214673E8648EC0D4D2AC2E6166FEE69
notBefore=Oct 12 07:03:39 2026 GMT
notAfter=Oct 12 07:03:39 2027 GMT
RSA key ok

[stderr]
```



Esecuzione registrata su chuck il 2026-10-12T07:03:39.405+00:00; exit status 0. Inizializzazione della cartella demoCA, generazione della chiave privata e creazione del certificato radice X.509 cacert.pem.

3.2 Richiesta ed emissione di certificati digitali (Esercizio 3.2)

Idea Generale e Contesto Concettuale: Il ciclo di vita di un certificato per un utente o un server si articola in passaggi formali:

1. **Generazione chiavi:** l'utente genera la propria coppia di chiavi privata/pubblica. La chiave privata non deve **mai** lasciare la macchina dell'utente!
2. **Creazione CSR (Certificate Signing Request):** viene creata una richiesta PKCS#10 contenente i dati identificativi (*Distinguished Name: CN, O, C*) e la chiave pubblica dell'utente. La CSR è firmata con la chiave privata dell'utente per dimostrare la prova di possesso (*Proof-of-Possession*).
3. **Validazione ed Emissione:** la CA verifica l'identità del richiedente, firma la CSR con la propria chiave privata K_{CA}^{-} e genera il certificato finale X.509, registrandone il numero di serie in `index.txt`.

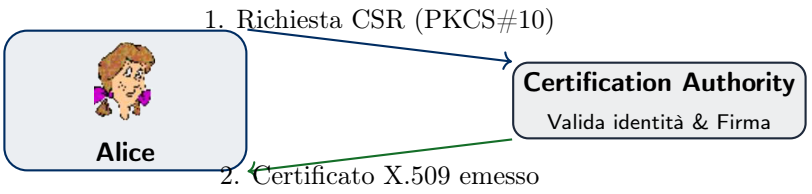


Figura 8: Flusso di richiesta ed emissione certificato tra utente e CA.

Esecuzione dei Comandi e Riscontro dei Risultati: Viene generata la richiesta di certificato CSR per Alice e successivamente emesso e verificato il certificato finale firmato dalla CA di laboratorio.

Chuck — Esercizio 3.2: Creazione CSR ed Emissione Certificato

```
$ set -e; cat > ca.cnf <<'EOF'
```

Esecuzione registrata su chuck il 2026-10-12T07:03:39.649+00:00; exit status 0. Generazione della richiesta CSR firmata da Alice ed emissione del certificato X.509 firmato dalla CA con aggiornamento del database.

3.3 Revoca di certificati (Esercizio 3.3)

Idea Generale e Contesto Concettuale: Un certificato emesso è normalmente valido fino alla data di scadenza. Tuttavia, se la chiave privata viene compromessa, smarrita o l'affiliazione dell'utente cessa, il certificato deve essere invalidato prima del termine. Questo avviene tramite la **revoca**:

- La CA inserisce il numero di serie del certificato nel proprio registro dei certificati revocati (`index.txt`).
- La CA genera ed emette periodicamente una **Certificate Revocation List (CRL)**, firmata digitalmente con la chiave privata della CA.
- I verificatori (Bob) devono scaricare la CRL e verificare il certificato con il controllo attivo (`-crl_check`).

Concetto fondamentale d'esame: una firma digitale apposta con una chiave privata associata a un certificato revocato rimane matematicamente verificabile, ma l'applicazione **deve rifiutarla** poiché il certificato associato non è più attendibile!

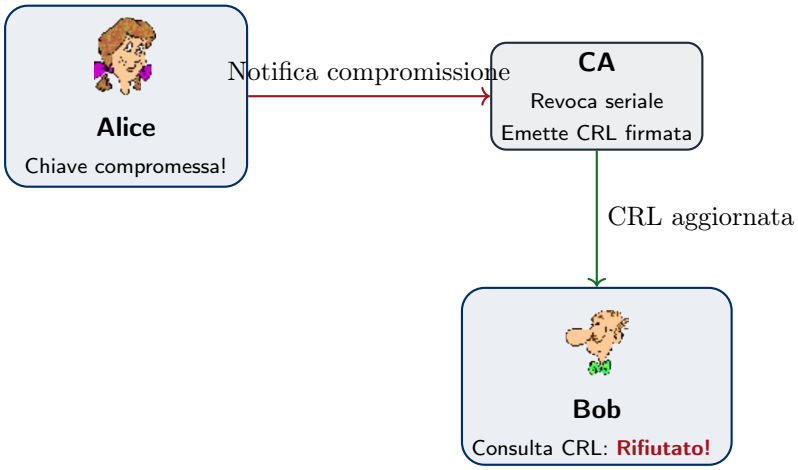


Figura 9: Ciclo di vita della revoca: la CA pubblica la CRL e Bob rigetta il certificato revocato.

Esecuzione dei Comandi e Riscontro dei Risultati: Viene revocato il certificato di Alice con `openssl ca -revoke`, generata la CRL e verificato che OpenSSL restituisca l'errore 23 (`certificate revoked`).

Chuck — Esercizio 3.3: Revoca del Certificato e Controllo CRL

```
$ set -e; openssl x509 -in alice.x509cert.pem -pubkey -noout > alice.pubkey.pem; openssl dgst -sha256 -sign
    alice.rsakey.pem -out signed-after-revoke.sig signed-message; openssl dgst -sha256 -verify alice.pubkey
    .pem -signature signed-after-revoke.sig signed-message; set +e; openssl verify -crl_check -CRLfile crl.
    pem -CAfile demoCA/cacert.pem alice.x509cert.pem; echo "certificate_validation_after_revoke_status:$?"

Verified OK
certificate_validation_after_revoke_status:2

[stderr]
C=IT, ST=Piemonte, O=Polito, OU=Students, CN=Alice
error 23 at 0 depth lookup: certificate revoked
error alice.x509cert.pem: verification failed
```

Esecuzione registrata su chuck il 2026-10-12T07:03:39.998+00:00; exit status 0. Il comando openssl verify con flag -crl_check fallisce con codice 23 segnalando esplicitamente lo stato di revoca.

4 Approfondimenti ed esercizi opzionali

4.1 Autenticazione (Esercizio 4.1)

Idea Generale e Contesto Concettuale (Challenge-Response): L'autenticazione basata su password è esposta a sniffing e dizionario. L'autenticazione a chiave pubblica si basa sul meccanismo **Challenge-Response**:

1. Il verificatore (Bob) genera una sfida casuale e imprevedibile chiamata **nonce** R (*number used once*).
2. Bob cifra il nonce con la chiave pubblica di Alice ($C = \text{Enc}_{K_A^+}(R)$) oppure chiede ad Alice di firmarlo.
3. Alice usa la propria chiave privata K_A^- per decifrare il nonce e lo invia a Bob (o invia la firma $\text{Sign}_{K_A^-}(R)$).
4. Bob confronta il valore ricevuto con R. Poiché solo chi possiede K_A^- può completare l'operazione, Alice è autenticata con certezza, e un attaccante non può riutilizzare vecchi messaggi (*anti-replay*).

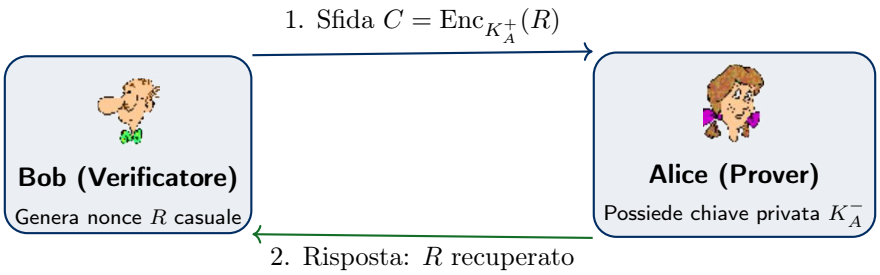


Figura 10: Protocollo Challenge-Response per autenticazione asimmetrica robusta.

Esecuzione dei Comandi e Riscontro dei Risultati: Viene generato un nonce casuale da 32 byte con `openssl rand`, cifrato con RSA-OAEP e la chiave pubblica, decifrato con la chiave privata e verificata l'esatta corrispondenza con `cmp`.

Chuck — Esercizio 4.1: Dimostrazione Challenge-Response RSA

```
$ set -e; openssl rand -out random 32; openssl pkeyutl -encrypt -pubin -inkey rsa.pubkey.alice -in random -
    out challenge -pkeyopt rsa_padding_mode:oaep; openssl pkeyutl -decrypt -inkey rsa.key.alice -in
    challenge -out response -pkeyopt rsa_padding_mode:oaep; cmp response random; echo "Challenge response
    matched"

Challenge response matched
```

Esecuzione registrata su chuck il 2026-10-12T07:03:39.872+00:00; exit status 0. Generazione del nonce casuale, cifratura asimmetrica OAEP, decifratura con chiave privata e validazione byte per byte.

4.2 Validazione di messaggi firmati (Esercizio 4.2)

Idea Generale e Analisi Pedagogica: Un punto critico della validazione crittografica riguarda la dimensione temporale:

- **Problema:** se un documento è firmato oggi e la chiave privata del firmatario viene compromessa tra sei mesi, come possiamo verificare se la firma è stata apposta *prima* della compromissione o *dopo*?
- Un semplice timestamp inserito nel testo del documento è facilmente manipolabile da chiunque conosca la chiave privata compromessa.
- **Soluzione architetturale:** è indispensabile utilizzare una **Time Stamping Authority (TSA)** fidata (RFC 3161), che appone una marca temporale firmata crittograficamente sul digest del documento al momento della firma.

In assenza di marca temporale fidata, se il certificato risulta revocato al momento del controllo, l'applicazione deve considerare la firma non attendibile.

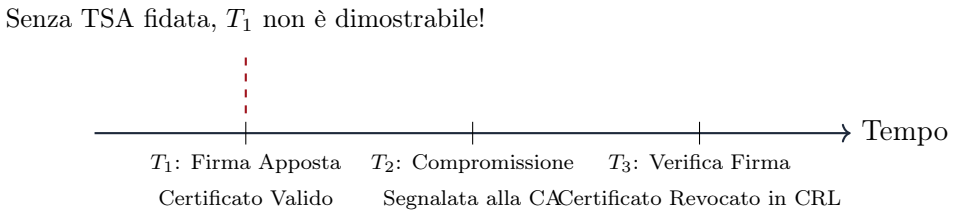


Figura 11: Linea temporale della validazione di firme digitali e rischio di compromissione retroattiva.

4.3 Progettazione di protocolli (Esercizio 4.3)

4.3.1 Protezione di messaggio (Esercizio 4.3.1)

Idea Generale e Requisiti di Progetto: Per proteggere un singolo messaggio trasmesso in modo asincrono (es. email o messaggistica memorizzata):

- **Confidenzialità:** garantita da cifratura ibrida con AEAD (AES-GCM) per evitare falle di integrità.
- **Autenticità e Integrità:** garantite da firma digitale asimmetrica dell'intero payload.
- **Ordine delle operazioni:** lo standard raccomanda *Sign-then-Encrypt* con inclusione esplicita dell'identità del destinatario all'interno del testo firmato (per prevenire attacchi di re-invoicing o inoltro malevolo a terzi), oppure contenitori standard CMS/PKCS#7.



Figura 12: Architettura di protezione a livello di messaggio: firma interna e cifratura esterna.

4.3.2 Protezione di canale (Esercizio 4.3.2)

Idea Generale e Requisiti di Progetto: A differenza della protezione a livello di messaggio, la **protezione di canale** (come TLS) protegge una sessione di comunicazione bidirezionale interattiva:

- **Perfect Forward Secrecy (PFS):** le chiavi di sessione vengono negoziate con scambi effimeri Diffie-Hellman (ECDHE); la compromissione futura della chiave a lungo termine del server non permette di decifrare il traffico passato.
- **Anti-replay e ordinamento:** garantiti da numeri di sequenza impliciti a livello di record TLS.
- **Autenticazione reciproca (mTLS):** opzionalmente il server richiede al client di presentare un certificato valido.

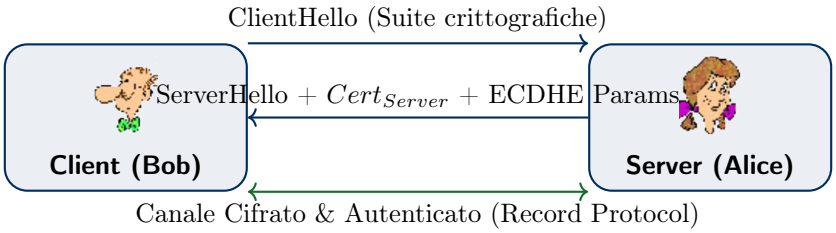


Figura 13: Instaurazione di un canale di comunicazione protetto a livello di trasporto (TLS).

4.4 Formati PKCS#7 (Esercizio 4.4)

Idea Generale e Contesto Concettuale: Gli standard **PKCS#7** (evoluto nello standard IETF Cryptographic Message Syntax - CMS, RFC 5652) definiscono formati standard per pacchetti crittografici:

- **SignedData:** contiene il contenuto, l'algoritmo di digest, la firma digitale e opzionalmente i certificati per facilitare la verifica da parte del destinatario.
- **EnvelopedData:** contiene dati cifrati simmetricamente e la chiave simmetrica protetta per uno o più destinatari tramite le loro chiavi pubbliche.
- **PKCS#12 (PFX):** formato contenitore protetto da password, utilizzato per archiviare in sicurezza una chiave privata congiuntamente alla catena di certificati associati.
- **Codifiche DER e PEM:** DER (*Distinguished Encoding Rules*) è la rappresentazione binaria ASN.1; PEM (*Privacy Enhanced Mail*) è la serializzazione testuale Base64 delimitata dalle intestazioni `---BEGIN...---`.

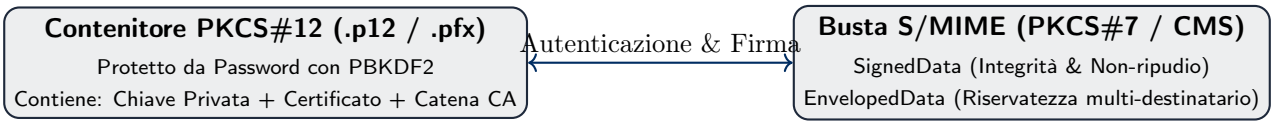


Figura 14: Struttura dei contenitori crittografici standard PKCS#12 e PKCS#7 / CMS.

Esecuzione dei Comandi e Riscontro dei Risultati: Vengono eseguite la creazione e la verifica di buste S/MIME firmate, la conversione tra formati DER e PEM, e l'esportazione protetta in contenitore PKCS#12 con `openssl pkcs12`.

Chuck — Esercizio 4.4: Buste S/MIME e Keystore PKCS#12

```
$ set -e; printf "PKCS7 laboratory payload\n" > msg; openssl smime -sign -nodetach -in msg -out p7snd.pem -
    outform PEM -signer alice.x509cert.pem -inkey alice.rsakey.pem -certfile demoCA/cacert.pem; openssl
    smime -verify -in p7snd.pem -inform PEM -CAfile demoCA/cacert.pem -out recovered.txt; tr -d "\r" <
    recovered.txt > recovered.normalized; cmp msg recovered.normalized; openssl smime -encrypt -aes-128-cbc
     -in msg -out p7e.pem -outform PEM alice.x509cert.pem; openssl smime -decrypt -in p7e.pem -inform PEM -
    inkey alice.rsakey.pem -out recovered-enveloped.txt; tr -d "\r" < recovered-enveloped.txt > recovered-
    enveloped.normalized; cmp msg recovered-enveloped.normalized; openssl x509 -in alice.x509cert.pem -
    outform DER -out alice.der; openssl x509 -inform DER -in alice.der -outform PEM -out alice-roundtrip.
    pem; openssl verify -CAfile demoCA/cacert.pem alice-roundtrip.pem; openssl pkcs12 -export -in alice.
    x509cert.pem -inkey alice.rsakey.pem -name "Alice Lab" -certfile demoCA/cacert.pem -out alice.p12 -
    passout pass:lab-pass; openssl pkcs12 -in alice.p12 -info -noout -passin pass:lab-pass

alice-roundtrip.pem: OK

[stderr]
Verification successful
MAC: sha256, Iteration 2048
MAC length: 32, salt length: 16
PKCS7 Encrypted data: PBES2, PBKDF2, AES-256-CBC, Iteration 2048, PRF hmacWithSHA256
Certificate bag
Certificate bag
PKCS7 Data
Shrouded Keybag: PBES2, PBKDF2, AES-256-CBC, Iteration 2048, PRF hmacWithSHA256
```

Esecuzione registrata su chuck il 2026-10-12T07:04:07.830+00:00; exit status 0. Generazione della busta firmata S/MIME, verifica tramite la CA fidata, decifratura e packaging sicuro della chiave privata in PKCS#12.